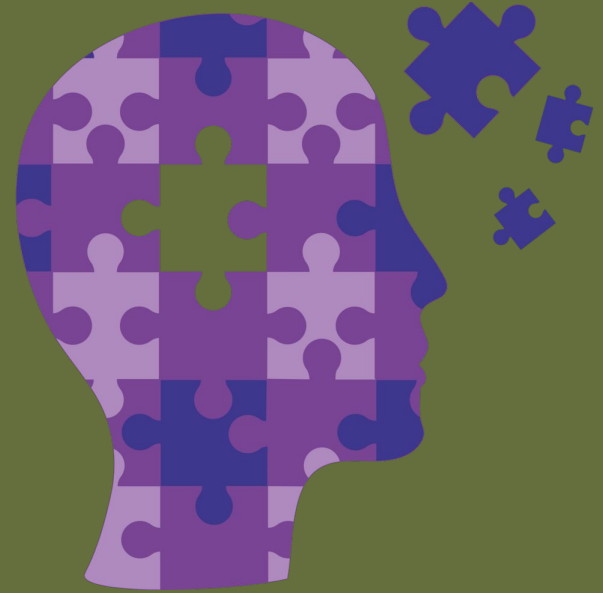


Alzheimer's Disease Research



Mahdi Moghaddami

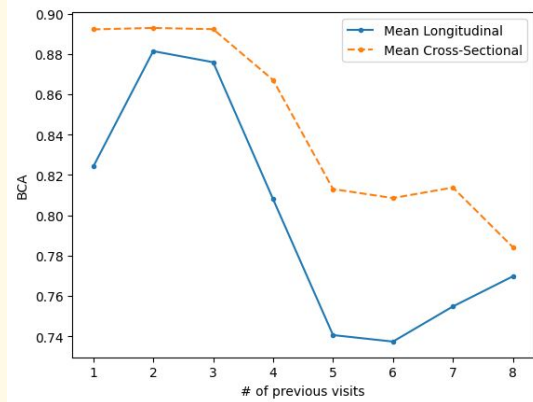
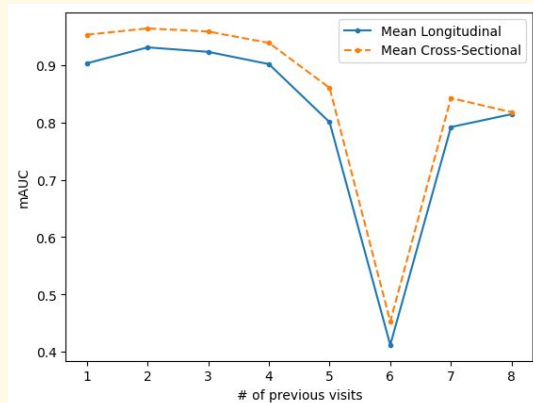
Brett Piggott

Projects Overview

Longitudinal vs. Cross-Sectional

- Longitudinal Models:
 - RNN
 - LSTM
 - GRU
 - MinimalRNN
- Cross-Sectional Models:
 - Random Forest
 - SVM
 - Logistic Regression

- This study compares two categories of modeling, longitudinal and cross-sectional, for predicting the clinical diagnosis of cognitive impairment in Alzheimer's disease.



Review Article

- Two major questions:
 1. How is longitudinal data used in AD?
 2. How should we write review articles when there is an abundance of papers?
 - ASReview
 - Elicit
 - Covidence
 - DistillerSR
 - Nested-Knowledge
- 2020–2026 papers
- Four sources:
 1. ACM Digital Library
 2. IEEEExplore
 3. Scopus
 4. PubMed
- We filter papers based on the existence of certain keywords in the title and abstract.
- We need further ranking of paper. We add different metric for papers:
 - Journal papers:
 - Journal Impact Factor (JIF)
 - CiteScore
 - SCImago Journal Rank (SJR)
 - Source Normalized Impact per Paper (SNIP)
 - Conference papers:
 - Core Ranking (A*, A, B, C)
 - CCF Ranking (A, B, C)
 - Both:
 - Google Scholar h5-index

AD Progression Prediction using DTI

- The question:
 - Will a subject progress to a worse stage of AD (CN → MCI or MCI → AD) in the next visit?
 - Stable/Converter prediction

- 337 subjects
- The data:
 - DTI
 - Structured clinical and biomarker data
- Brett will talk more about this project!

AD Progression Prediction using fMRI

What is fMRI?

- Functional Magnetic Resonance Imaging is a type of brain scan that shows both the structure of the brain and, more importantly, **which areas are active during specific tasks or at rest.**
- It's acquired by tracking blood flow changes. When a brain area is more active, it uses more oxygen, and fMRI can detect that via something called the **BOLD signal** (Blood Oxygen Level Dependent signal).
- Each voxel contains a BOLD signal that changes over time.
- It's basically **a number of 3D images.**



Preprocessing Steps using fMRIPrep

1. Convert the data into a standard format:
 - DICOM (.dcm) → NIfTI (.nii)
2. Motion Correction:
 - During a several-minute fMRI scan, the subject inevitably moves a little.
 - fMRIPrep estimates how each volume moved relative to a reference volume.
 - It then realigns the volumes.
3. Slice timing correction
 - An fMRI volume isn't actually acquired all at once.
 - The scanner acquires different slices at slightly different times.
 - Slice-timing correction estimates what the signal would have been at the appropriate timing.
4. Susceptibility distortion correction
 - Magnetic fields get warped near air-filled spaces in the skull (like the sinuses).
 - This step calculates that magnetic warping and "un-squishes" the image back to its true anatomical shape.
5. Coregistration:
 - fMRI images are low-resolution and blurry.
 - To know exactly where the brain activity is happening, fMRIPrep aligns the functional image with the subject's anatomical image (T1w MRI).
6. Spatial Normalization:
 - Every person's brain is a slightly different size and shape.
 - fMRIPrep estimates a nonlinear transformation that maps the individual's anatomy to a *standard* brain template (usually called MNI space).

Generating the Brain Functional Connectivity Network using NiLearn

1. Parcellation (Applying an Atlas):

- Instead of analyzing 100,000 tiny individual voxels, Nilearn uses a brain "atlas" to group those voxels into larger, meaningful ROIs (e.g., the hippocampus, the amygdala, the motor cortex).

2. Extracting Time Series (4D $\rightarrow$ Regions $\times$ Time)

- Suppose a region contains $v=|V|$ voxels, each voxel has its own BOLD time series. We summarize these voxel signals, typically by taking the mean signal across voxels in the region. We get: $Region1=[x1,x2,x3,...,xT]$ where T is the number of fMRI time points.
- We do this for every region.

2. Correlation (Regions $\times$ Time $\rightarrow$ Region $\times$ Region):

- "How similarly does Region A's activity fluctuate with Region B's activity?"
- Suppose:
 - $R1 = [0.2, 0.4, 0.1, -0.2, -0.4, \dots]$
 - $R2 = [0.3, 0.5, 0.2, -0.1, -0.5, \dots]$
- We can calculate their Pearson correlation:
 - $r_{1,2} = \text{corr}(R1, R2)$
- The final result is our 2D Matrix

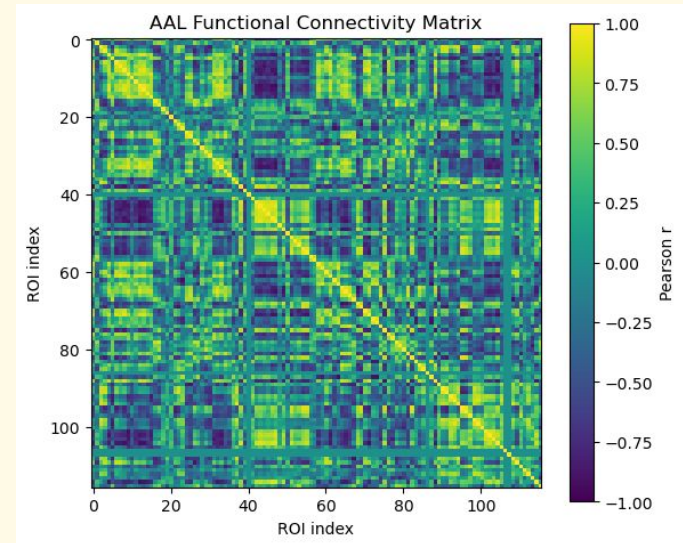
$$r = \frac{\sum (x - m_x)(y - m_y)}{\sqrt{\sum (x - m_x)^2 \sum (y - m_y)^2}}$$

- Where x, y : Two numeric vectors of the same length
- m_x, m_y : Mean values of x and y respectively
- $-1 \leq r \leq 1$

FC Matrix

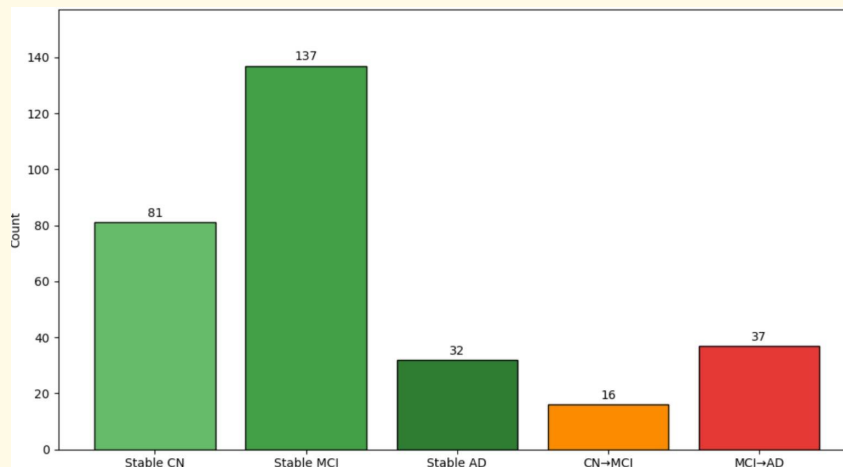
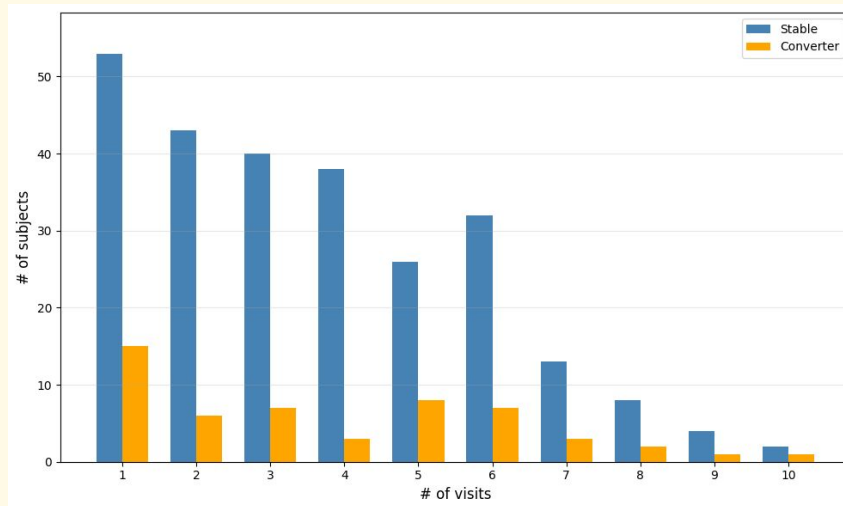
- We calculate $r_{i,j}$ for every i and j ($1 \leq i, j \leq N$) where N =number of ROIs in our atlas.

	1	2	3	4
1	1	.72	.15	-.10
2	.72	1	.43	.05
3	.15	.43	1	.61
4	-.10	.05	.61	1

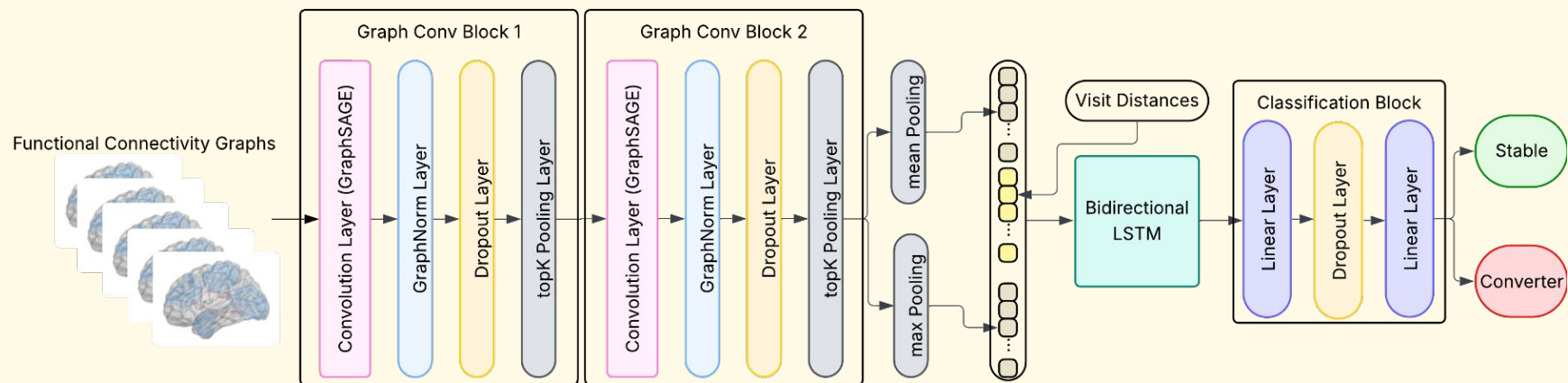


The Dataset

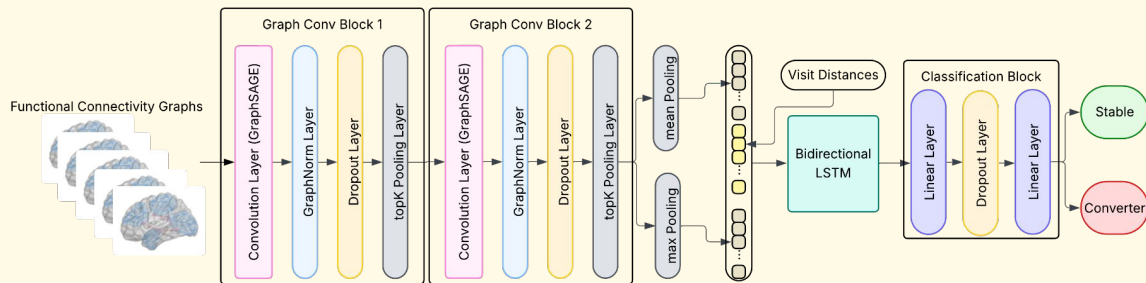
- **Stable sequence:** last visit dx same as previous.
- **Converter sequence:** final visit is a conversion



The Model



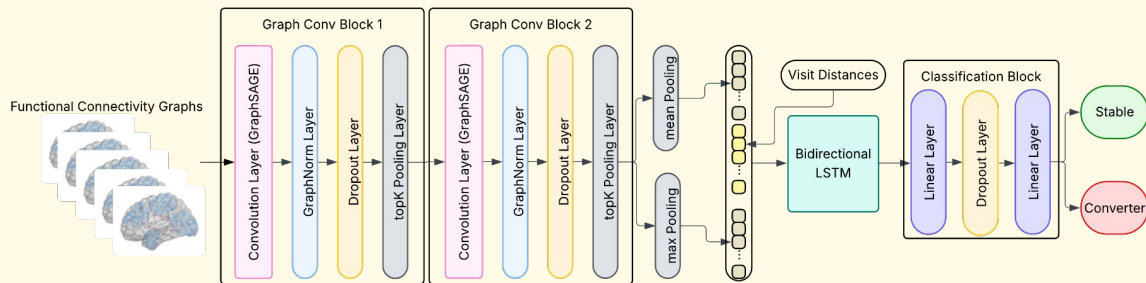
Input



- The original 2D functional connectivity matrix is shaped like: (ROIs x ROIs)
 - For example, if there are 100 ROIs: (100, 100)
- For the GNN input, that matrix is converted into a graph:
 - nodes = ROIs
 - edges = functional connections between ROIs
 - An edge exists only if the correlation between two nodes is $\geq \text{threshold}$.
 - node features = connectivity profile of each ROI (its row of the FC matrix)
- The key difference is:
 - The 2D matrix is a dense table of pairwise correlations.
 - The GNN input is a graph object built from that matrix, where ROIs are nodes and connections are edges, letting graph convolutions aggregate information over neighboring ROIs.
- The model input is multiple graphs for one subject, arranged as a visit sequence.

GraphSAGE

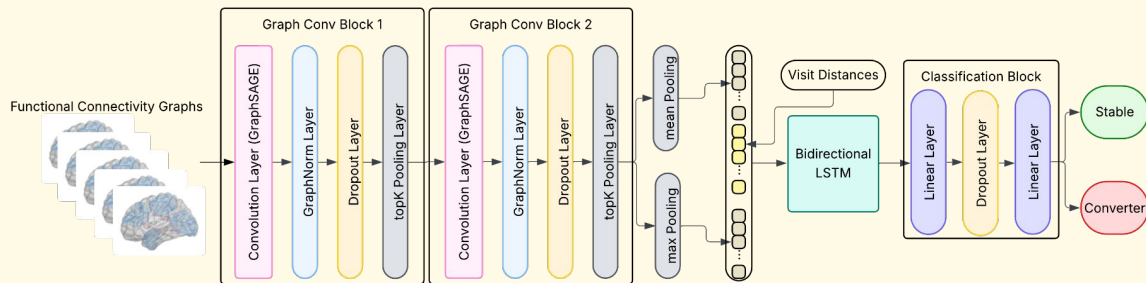
[NeurIPS 2017]



- Detailed explanation.
- [Link1](#)
- [Link2](#)
- For each node i :
 - $x_i = [\text{corr}(i, R1), \text{corr}(i, R2), \dots, \text{corr}(i, R100)]$
 - We average the neighbor features:
 - $m_i = \text{mean}(\{x_j : j \in N(i)\})$
 - Then combine the node's own features with that neighbor average:
 - $h_i = \sigma(W_{\text{self}} x_i + W_{\text{neigh}} m_i + b)$
- h_i is the new embedding for node i
- W_{self} is a learned weight matrix for the node itself
- W_{neigh} is a learned weight matrix for its neighbors
- b is a learned bias
- σ is an activation function
- GraphSAGE does this for all N nodes:
 - So the output of the first GraphSAGE layer is:
 $H \in \mathbb{R}^{(N \times \text{hidden_dim})}$
 - $H = [h1, h2, \dots, hN]$

GraphNorm

[ICML 2021]



- Input: $H = [h_1, h_2, \dots, h_N] \in \mathbb{R}^{(N \times \text{hidden_dim})}$
- GraphNorm normalizes the node embeddings within that one graph. In simple terms, it asks:
 - For this subject's visit, are some node features unusually large or small compared with the rest of the graph?
 - Then it rescales them so the graph is easier for the next layers to process.
- Output: $H' = [h'_1, h'_2, \dots, h'_N] \in \mathbb{R}^{(N \times \text{hidden_dim})}$

- Each node is normalized:

$$\hat{h}_{i,k} = \frac{h_{i,k} - \mu_k}{\sigma_k}$$

- Where

$$\mu_k = \frac{1}{N} \sum_{i=1}^N h_{i,k}$$

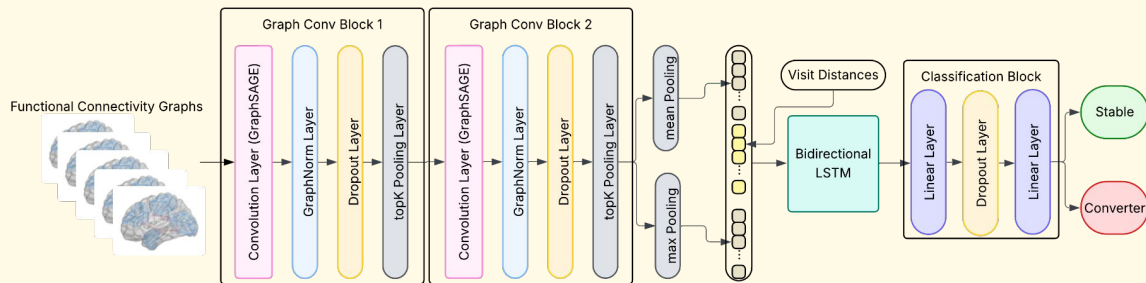
$$\sigma_k = \sqrt{\frac{1}{N} \sum_{i=1}^N (h_{i,k} - \mu_k)^2 + \epsilon}$$

- GraphNorm also learns a small rescaling and shifting for each feature:

$$h'_{i,k} = \gamma_k \hat{h}_{i,k} + \beta_k$$

- Where beta and gamma are learned parameters.

Dropout



- Dropout randomly turns off some feature values during training.
- Input: $H' = [h1', h2', \dots, hN'] \in R^{(N \times \text{hidden_dim})}$
- For each node i and feature k , we have a mask:
 - $m_{i,k} = \text{Bernoulli}(1-p)$
 - p : dropout probability

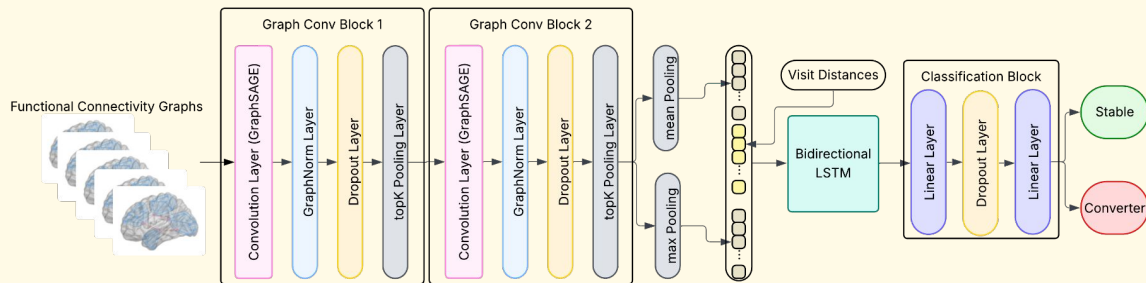
$$\tilde{h}_{i,k} = \frac{m_{i,k} h'_{i,k}}{1 - p}$$

- So:

$$\tilde{H} = \frac{M \odot H'}{1 - p}$$

- Output: $H'' = [h1'', h2'', \dots, hN''] \in R^{(N \times \text{hidden_dim})}$

TopK Pooling



- Input: $H'' = [h1'', h2'', \dots, hN''] \in \mathbb{R}^{(N \times \text{hidden_dim})}$
- TopK pooling keeps only the "most important" nodes and removes the rest.
 - In simple terms, the model learns a scoring rule for each ROI. ROIs with higher scores are kept; ROIs with lower scores are dropped.

- First, TopK pooling learns a projection vector $p \in \mathbb{R}^{\text{hidden_dim}}$

- Then each node i gets a score:

$$s_i = \frac{\tilde{h}_i^\top p}{\|p\|}$$

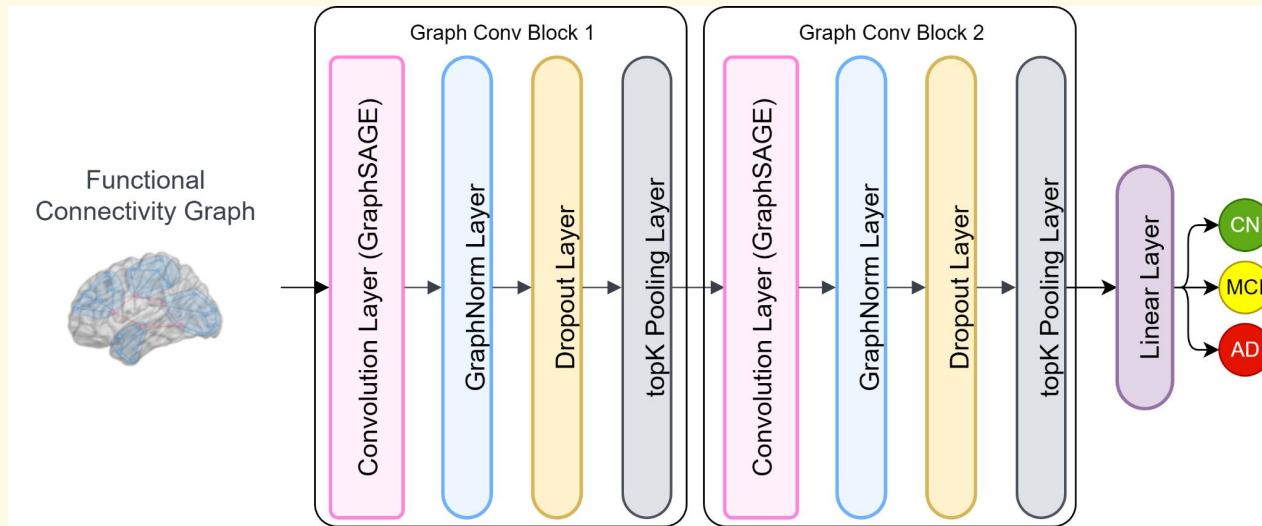
- Then the model keeps only the top K nodes.
 - $K = \lceil rN \rceil$
 - Here r is the pooling ratio. (we use $r=0.3$)
- For the kept nodes, TopK pooling also gates their embeddings using the score:

$$h_i^{\text{pool}} = \tilde{h}_i \cdot \tanh(s_i)$$

- Output: $H_{\text{pool}} \in \mathbb{R}^{(K \times \text{hidden_dim})}$

Pretraining

- Using 20% of data



Challenges

- Missing visit
 - Some visits are missing fMRI but have data for other modalities (sMRI, cognitive scores, PET, etc.)
-

Thank you!